

CSC 211: Computer Programming

Basic C++ Concepts and Syntax

Prof. Michael Conti

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Original design and development by Dr. Marco Alvarez

Administrative Notes

Administrative notes

- A00 - still pending
- Micro-Programming Challenge#01
- Once A00 opens, make sure you're in contact with group
 - ✓ If you fail to respond and your group moves forward without you, you won't be listed for credit in A00 submission

C++ Basics

Basics

- Everything in C++ is **case sensitive**
- Curly braces are used to denote **code blocks**

```
int main() {  
    // body ...  
}
```

- All statements end with a **semicolon** (can use multiple lines)

```
int a;  
a = 100;  
a = a + 111;
```

```
int a;  
a = 100;  
a = a  
+  
111;
```

The main function

```
int main () {  
    // body  
    return 0;  
}
```

```
int main (int argc, char *argv[]) {  
    // body  
    return 0;  
}
```

The `main` function

- Automatically called at program startup
 - ✓ designated entry point to a program that is executed in a hosted environment (operating system)
- Prototype cannot be modified
- Cannot be used anywhere in the program
 - ✓ cannot be overloaded
 - ✓ cannot be called recursively
- Its address cannot be taken

The `main` function

- Does not need to contain the `return` statement
 - ✓ if control reaches the end of `main` without encountering a `return` statement, the effect is that of executing `return 0;`
- Execution of the `return` (or the implicit `return`) is equivalent to:
 - ✓ leaving the function normally (which destroys local objects)
 - ✓ calling `std::exit` with the same argument as the argument of the `return`
 - ✓ `std::exit` destroys static objects and terminates the program

Comments

- **Comments** can be single-line or multi-line
 - ✓ comments are ignored by the compiler

```
int a;  
// ignore the following line  
// a = 100;  
a = 200;
```

```
int a;  
// ignore this block  
a = 100;  
/*  
a = a  
+  
111;  
*/
```

C++ keywords

This is a list of reserved keywords in C++. Since they are used by the language, these keywords are not available for re-definition or overloading.

<code>alignas</code> (since C++11)	<code>default</code> (1)	<code>register</code> (2)
<code>alignof</code> (since C++11)	<code>delete</code> (1)	<code>reinterpret_cast</code>
<code>and</code>	<code>do</code>	<code>requires</code> (since C++20)
<code>and_eq</code>	<code>double</code>	<code>return</code>
<code>asm</code>	<code>dynamic_cast</code>	<code>short</code>
<code>atomic_cancel</code> (TM TS)	<code>else</code>	<code>signed</code>
<code>atomic_commit</code> (TM TS)	<code>enum</code>	<code>sizeof</code> (1)
<code>atomic_noexcept</code> (TM TS)	<code>explicit</code>	<code>static</code>
<code>auto</code> (1)	<code>export</code> (1)(3)	<code>static_assert</code> (since C++11)
<code>bitand</code>	<code>extern</code> (1)	<code>static_cast</code>
<code>bitor</code>	<code>false</code>	<code>struct</code> (1)
<code>bool</code>	<code>float</code>	<code>switch</code>
<code>break</code>	<code>for</code>	<code>synchronized</code> (TM TS)
<code>case</code>	<code>friend</code>	<code>template</code>
<code>catch</code>	<code>goto</code>	<code>this</code>
<code>char</code>	<code>if</code>	<code>thread_local</code> (since C++11)
<code>char8_t</code> (since C++20)	<code>inline</code> (1)	<code>throw</code>
<code>char16_t</code> (since C++11)	<code>int</code>	<code>true</code>
<code>char32_t</code> (since C++11)	<code>long</code>	<code>try</code>
<code>class</code> (1)	<code>mutable</code> (1)	<code>typedef</code>
<code>compl</code>	<code>namespace</code>	<code>typeid</code>
<code>concept</code> (since C++20)	<code>new</code>	<code>typename</code>
<code>const</code>	<code>noexcept</code> (since C++11)	<code>union</code>
<code>constexpr</code> (since C++20)	<code>not</code>	<code>unsigned</code>
<code>constexpr</code> (since C++11)	<code>not_eq</code>	<code>using</code> (1)
<code>constinit</code> (since C++20)	<code>nullptr</code> (since C++11)	<code>virtual</code>
<code>const_cast</code>	<code>operator</code>	<code>void</code>
<code>continue</code>	<code>or</code>	<code>volatile</code>
<code>co_await</code> (since C++20)	<code>or_eq</code>	<code>wchar_t</code>
<code>co_return</code> (since C++20)	<code>private</code>	<code>while</code>
<code>co_yield</code> (since C++20)	<code>protected</code>	<code>xor</code>
<code>decltype</code> (since C++11)	<code>public</code>	<code>xor_eq</code>
	<code>reflexpr</code> (reflection TS)	

Identifiers

- Names given to entities such as data types, objects, references, variables, functions, macros, class members, data types, etc.
- **Identifiers** cannot be the same as any of the reserved words
- A valid **identifier** is a sequence of one or more letters, digits, and underscore characters
 - ✓ cannot begin with a digit
 - ✓ some compilers may impose limits on length (e.g. 2048 characters Microsoft C++)
- Examples:

Basic Data Types

▸ Void	void
▸ Boolean	bool
▸ Integer	int
▸ Floating Point	float, double
▸ Character	char

Variables

- A **variable** is a named location in memory
 - ✓ store values during program execution
 - ✓ memory location irrelevant (we use names for access)
- C++ type system keeps track of the size of the memory block and how to interpret its contents
- Declaration:
 - ✓ Parenthesis will initialize the values as well (optional)

```
<type> <identifier> [= <initializer>];  
      <type> <list of identifiers>;
```

Examples

Literals

- Tokens that represent constant values explicitly embedded in the source code
 - ✓ integers, characters, floating point, strings, boolean, user-defined
- Examples:

```
int a = 200;
```

```
true false
```

Escape Sequences

Escape sequence	Description	Representation
\'	single quote	byte 0x27 in ASCII encoding
\"	double quote	byte 0x22 in ASCII encoding
\?	question mark	byte 0x3f in ASCII encoding
\\	backslash	byte 0x5c in ASCII encoding
\a	audible bell	byte 0x07 in ASCII encoding
\b	backspace	byte 0x08 in ASCII encoding
\f	form feed - new page	byte 0x0c in ASCII encoding
\n	line feed - new line	byte 0x0a in ASCII encoding
\r	carriage return	byte 0x0d in ASCII encoding
\t	horizontal tab	byte 0x09 in ASCII encoding
\v	vertical tab	byte 0x0b in ASCII encoding
\nnn	arbitrary octal value	byte nnn
\Xnn	arbitrary hexadecimal value	byte nn
\Unnnn (since C++11)	universal character name (arbitrary Unicode value); may result in several characters	code point U+nnnn
\Unnnnnnnn (since C++11)	universal character name (arbitrary Unicode value); may result in several characters	code point U+nnnnnnnn

Statements

- Fragments of code that are executed in sequence
- Types of statements:
 - ✓ expression statements
 - ✓ compound statements
 - brace-enclosed sequences of statements
 - ✓ selection statements
 - ✓ iteration statements
 - ✓ jump statements
 - ✓ declaration statements
 - ✓ try blocks

Examples

```
int main() {  
    int n = 1;           // declaration  
    n = n + 1;           // expression  
    std::cout << "n = " << n << '\n'; // expression  
    return 0;           // jump  
}
```

```
if (x > 5)               // start of if statement  
{                       // start of block  
    int n = 1;           // declaration statement  
    std::cout << n;      // expression statement  
}                         // end of block, end of if statement
```

Expressions

- An **expression** is a sequence of operators and their operands
 - ✓ it can also be a literal or a variable name, etc.
- Expression evaluation may produce a result (has a type)
 - ✓ e.g., evaluation of **2+2** produces the result **4**
- Expression evaluation may generate side-effects
 - ✓ e.g., output of a **std::cout** expression

Arithmetic Expressions

Mathematical Formula

C++ Expression

$$b^2 - 4ac$$

`b*b - 4*a*c`

$$x(y + z)$$

`x*(y + z)`

$$\frac{1}{x^2 + x + 3}$$

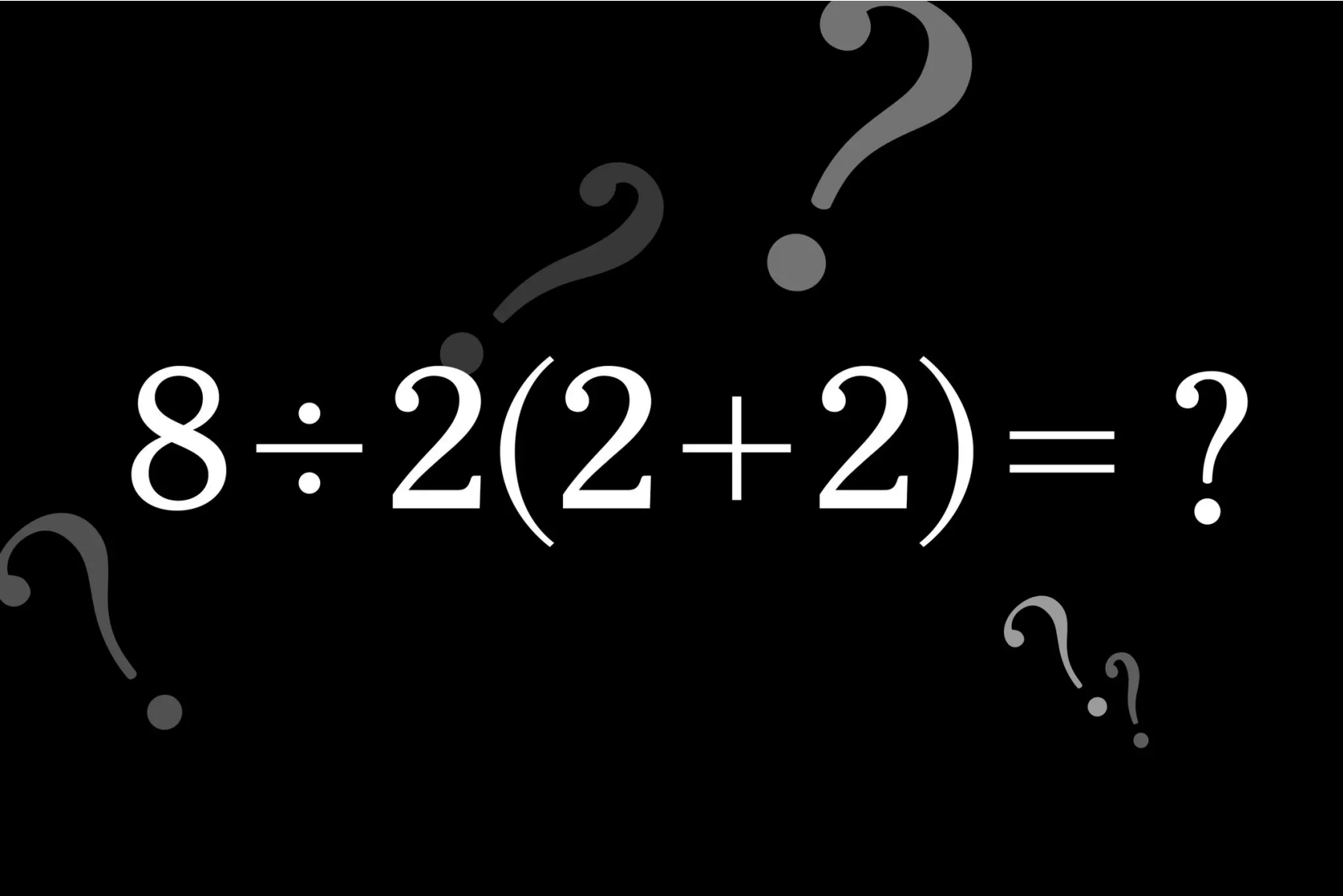
`1/(x*x + x + 3)`

$$\frac{a + b}{c - d}$$

`(a + b)/(c - d)`

Common operators						
assignment	increment decrement	arithmetic	logical	comparison	member access	other
<code>a = b</code> <code>a += b</code> <code>a -= b</code> <code>a *= b</code> <code>a /= b</code> <code>a %= b</code> <code>a &= b</code> <code>a = b</code> <code>a ^= b</code> <code>a <<= b</code> <code>a >>= b</code>	<code>++a</code> <code>--a</code> <code>a++</code> <code>a--</code>	<code>+a</code> <code>-a</code> <code>a + b</code> <code>a - b</code> <code>a * b</code> <code>a / b</code> <code>a % b</code> <code>~a</code> <code>a & b</code> <code>a b</code> <code>a ^ b</code> <code>a << b</code> <code>a >> b</code>	<code>!a</code> <code>a && b</code> <code>a b</code>	<code>a == b</code> <code>a != b</code> <code>a < b</code> <code>a > b</code> <code>a <= b</code> <code>a >= b</code> <code>a <=> b</code>	<code>a[b]</code> <code>*a</code> <code>&a</code> <code>a->b</code> <code>a.b</code> <code>a->*b</code> <code>a.*b</code>	<code>a(...)</code> <code>a, b</code> <code>? :</code>
Special operators						
<code>static_cast</code> converts one type to another related type <code>dynamic_cast</code> converts within inheritance hierarchies <code>const_cast</code> adds or removes <code>cv</code> qualifiers <code>reinterpret_cast</code> converts type to unrelated type <code>C-style cast</code> converts one type to another by a mix of <code>static_cast</code>, <code>const_cast</code>, and <code>reinterpret_cast</code> <code>new</code> creates objects with dynamic storage duration <code>delete</code> destructs objects previously created by the <code>new</code> expression and releases obtained memory area <code>sizeof</code> queries the size of a type <code>sizeof...</code> queries the size of a <code>parameter pack</code> (since C++11) <code>typeid</code> queries the type information of a type <code>noexcept</code> checks if an expression can throw an exception (since C++11) <code>alignof</code> queries alignment requirements of a type (since C++11)						

Operator Precedence / Associativity


$$8 \div 2(2 + 2) = ?$$

Operator Precedence / Associativity

- **Operator precedence** determines which operator is performed first in an expression with more than one operators with different precedence
- **Operators Associativity** is used when two operators of same precedence appear in an expression. Associativity can be either Left to Right or Right to Left.
- For example: ‘*’ and ‘/’ have the same precedence and their associativity is Left to Right, so the expression “100 / 10 * 10” is treated as “(100 / 10) * 10”.

Operator Precedence / Associativity

Precedence	Operator	Description	Associativity
1	::	Scope resolution	Left-to-right
2	a++ a-- type() type{} a() a[] . ->	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access	
3	++a --a +a -a ! ~ (type) *a &a sizeof co_await new new[] delete delete[]	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation	Right-to-left
4	.* ->*	Pointer-to-member	Left-to-right
5	a*b a/b a%b	Multiplication, division, and remainder	
6	a+b a-b	Addition and subtraction	
7	<< >>	Bitwise left shift and right shift	
8	<=>	Three-way comparison operator (since C++20)	
9	< <= > >=	For relational operators < and ≤ respectively For relational operators > and ≥ respectively	
10	== !=	For relational operators = and ≠ respectively	
11	&	Bitwise AND	
12	^	Bitwise XOR (exclusive or)	
13		Bitwise OR (inclusive or)	
14	&&	Logical AND	
15		Logical OR	

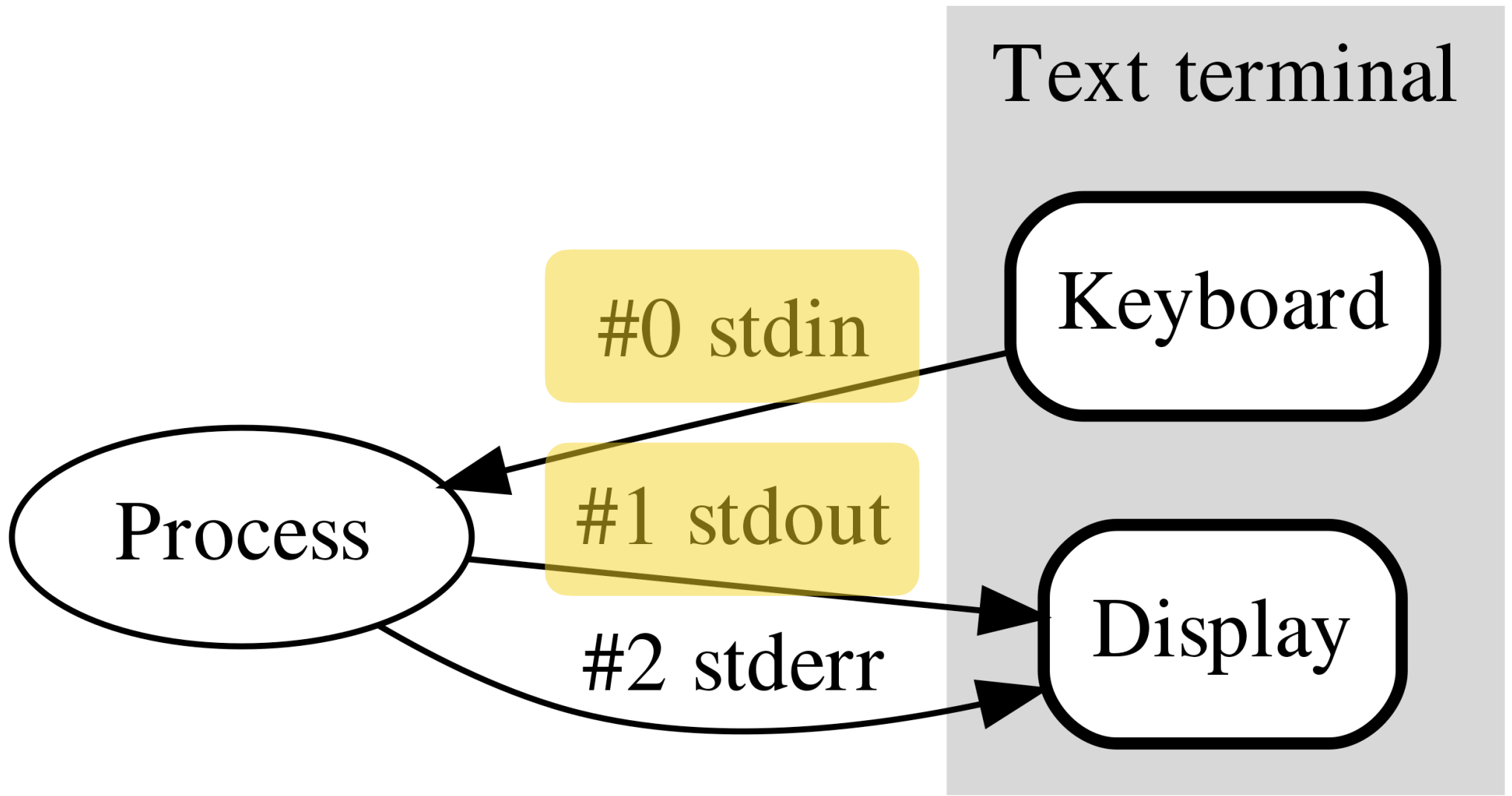
Operator Precedence / Associativity

16	<code>a?b:c</code> <code>throw</code> <code>co_yield</code> <code>=</code> <code>+= -=</code> <code>*= /= %=</code> <code><<= >>=</code> <code>&= ^= =</code>	Ternary conditional ^[note 2] throw operator yield-expression (C++20) Direct assignment (provided by default for C++ classes) Compound assignment by sum and difference Compound assignment by product, quotient, and remainder Compound assignment by bitwise left shift and right shift Compound assignment by bitwise AND, XOR, and OR	Right-to-left
17	<code>,</code>	Comma	Left-to-right

Basic Input/Output

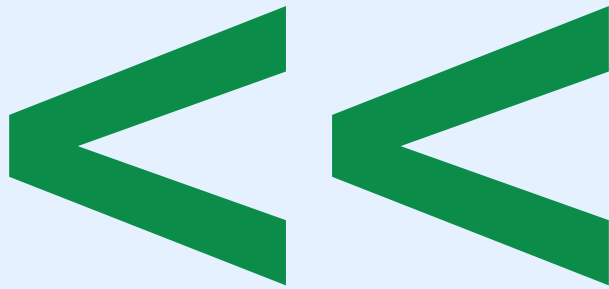
- Data streams are just sequences of data
- **Input Stream**
 - ✓ data passed to programs
 - ✓ typically originates from keyboard or files
- **Output Stream**
 - ✓ output from programs
 - ✓ typically goes to the terminal/monitor or files

Basic Input/Output



std::cout

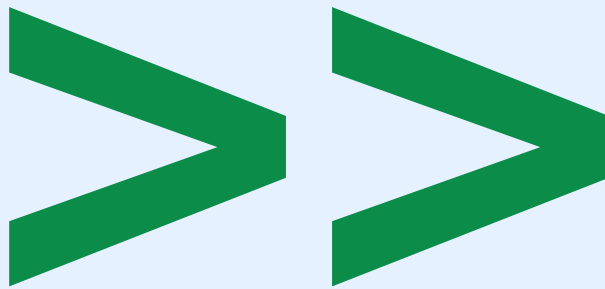
the output stream



the insertion operator

std::cin

the input stream



the extraction operator

Include directives

- Required to add **library** files to programs
- For standard **input** and **output** use:

#include <iostream>